



Business Central Agentic Development Framework

Simple Enough for Functional Consultants.
Robust Enough for Pro Developers.

Created by AJ Ansari, Microsoft MVP

The Playbook — four phases, a gate on every one

Built so a Functional Consultant can build AL the right way

This Agentic Development Framework was created to help Business Central Functional Consultants use AI to build AL extensions and apps the **right** way — though professional AL developers will find it just as useful.

Two editions: the **full framework** for substantial projects, and **Lite** for smaller ones that need to move fast. Both apply the same AL standards.

The spark was Microsoft's Business Central Agentic Engineering Process vision, first unveiled at Directions North America in April 2026.

Either way, the routine grounds the agent in explicit rules, official references and quality tooling rather than AI guesswork alone:

OnlyCopilotFans AL Development Standards Guide The AL rules the routine applies at each step, fetched in automatically.
AL MCP Server Build, publish, symbol and diagnostic tools, driven directly.
BC Base App on Microsoft Learn Official references instead of half-remembered platform behavior.
BCQuality A curated quality knowledge base, snapshotted per project.
OnlyCopilotFans BC AL Patterns A portable pattern library

Shapes and roles

SHAPE LEGEND

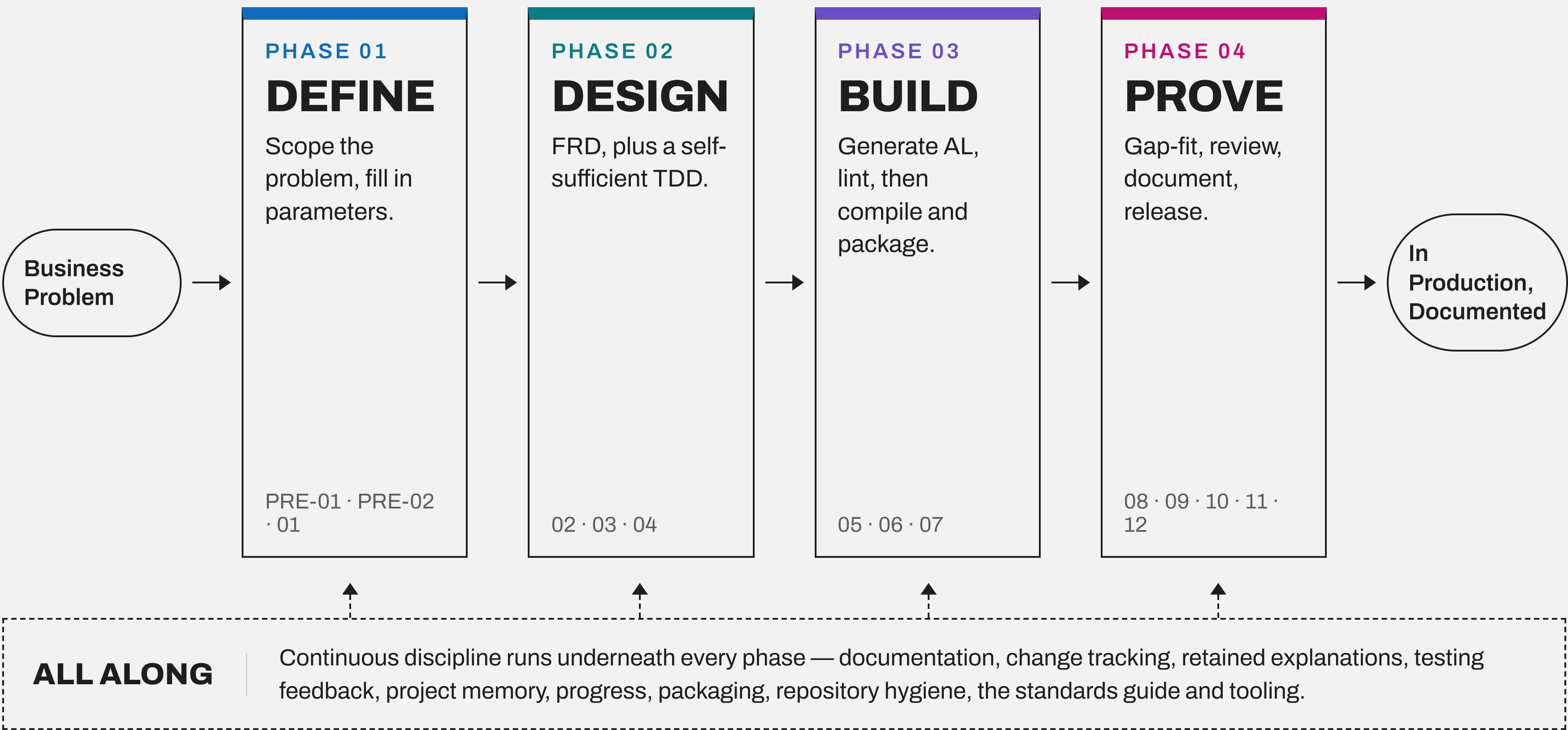
	Start or end point of a flow
	An action, step, or process
	Input or output — data, a document
	An exit gate — the checkpoint that must be met
	A decision point
	A cross-cutting concern, running underneath

ROLE LEGEND — §1.7

	Main role Generates and edits all code; owns the continuity documents.
	Light role Post-generation pre-flight only. Reports findings, never edits.
	Reasoning role Authors, reviews, diagnoses. Never edits code or continuity docs.

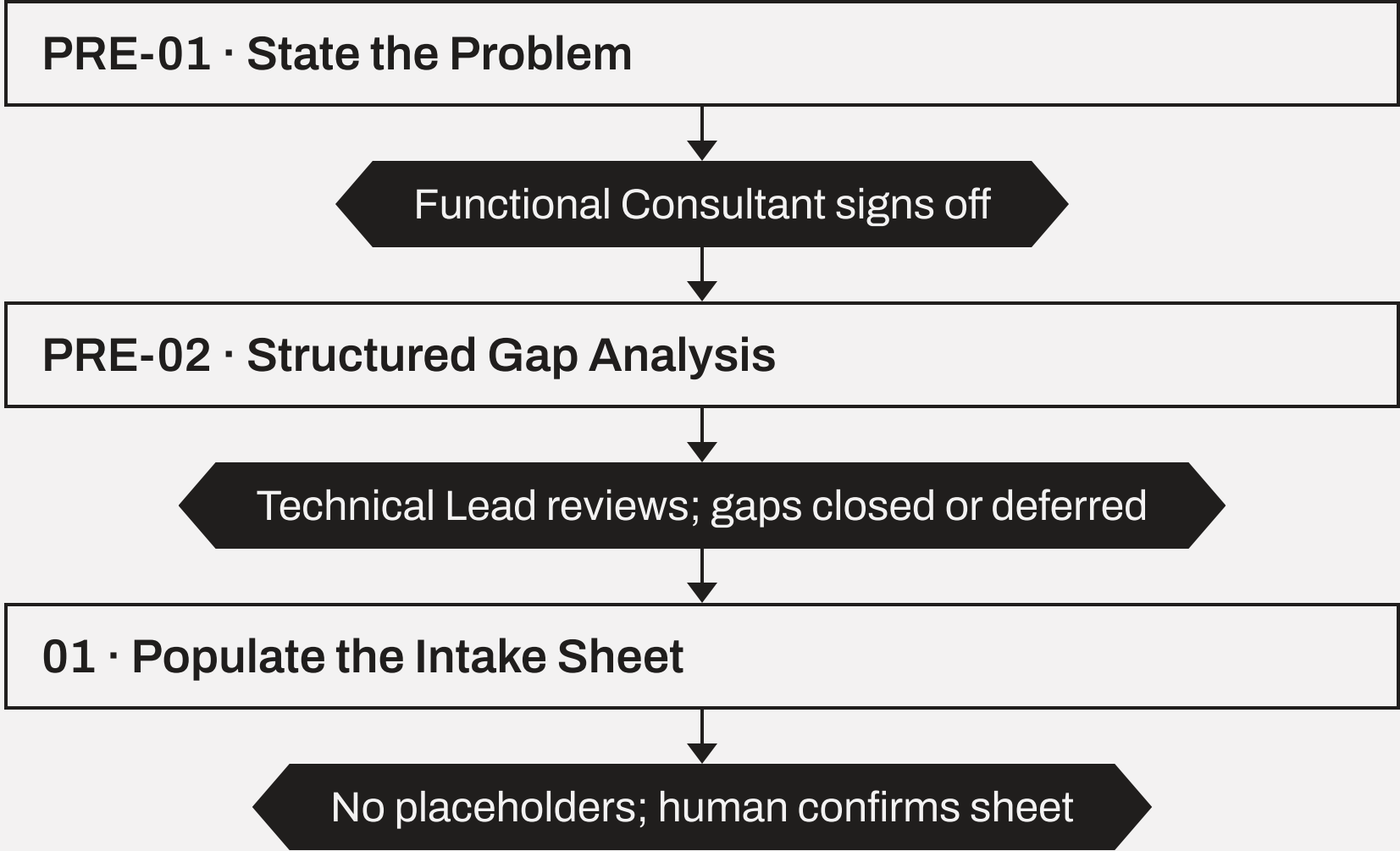
Where §1.7 role assignment isn't configured, every colored step is done by the single running model instead.

The whole framework in one picture



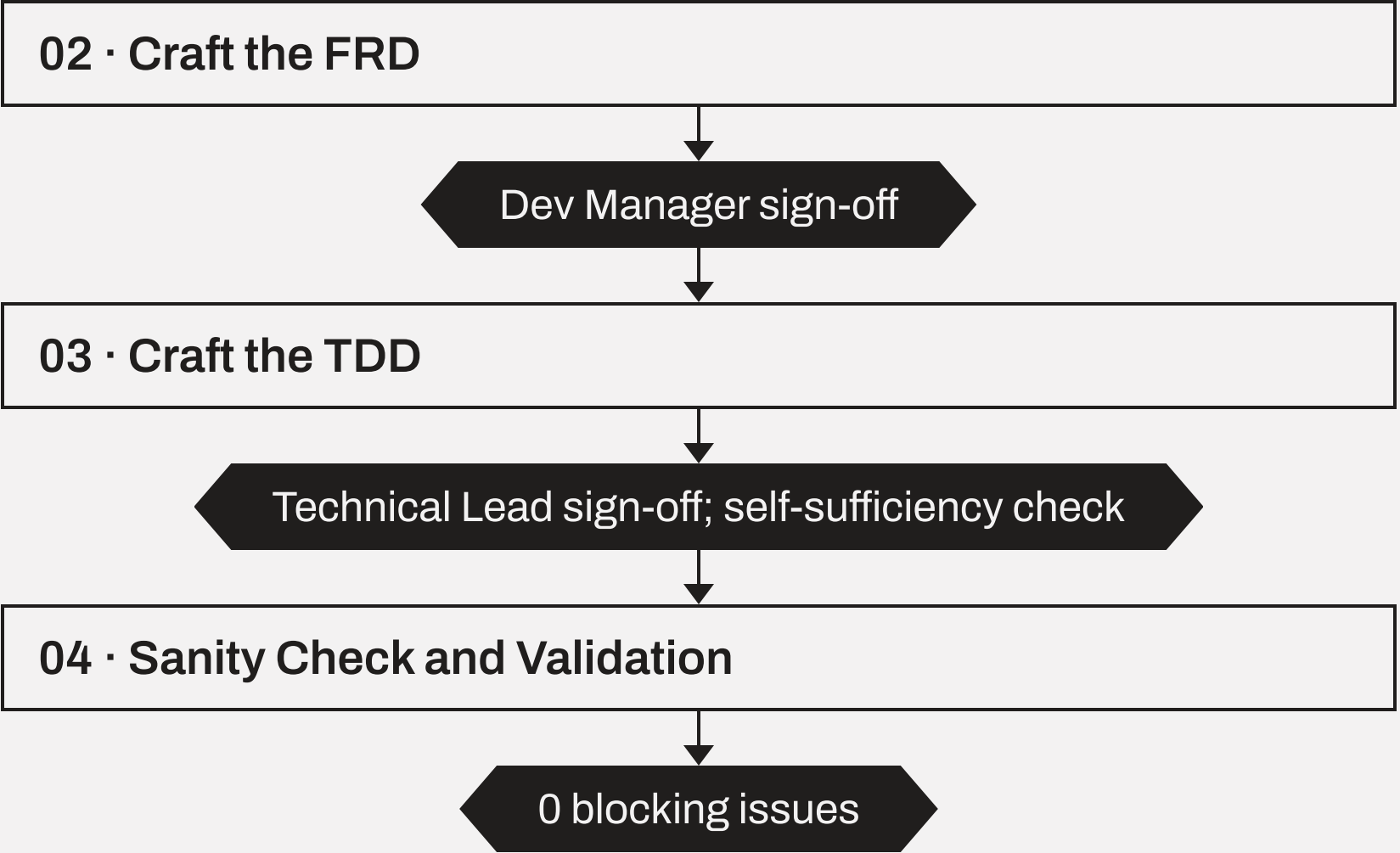
Every step, and the gate that closes it

DEFINE



↓ to DESIGN, Step 02

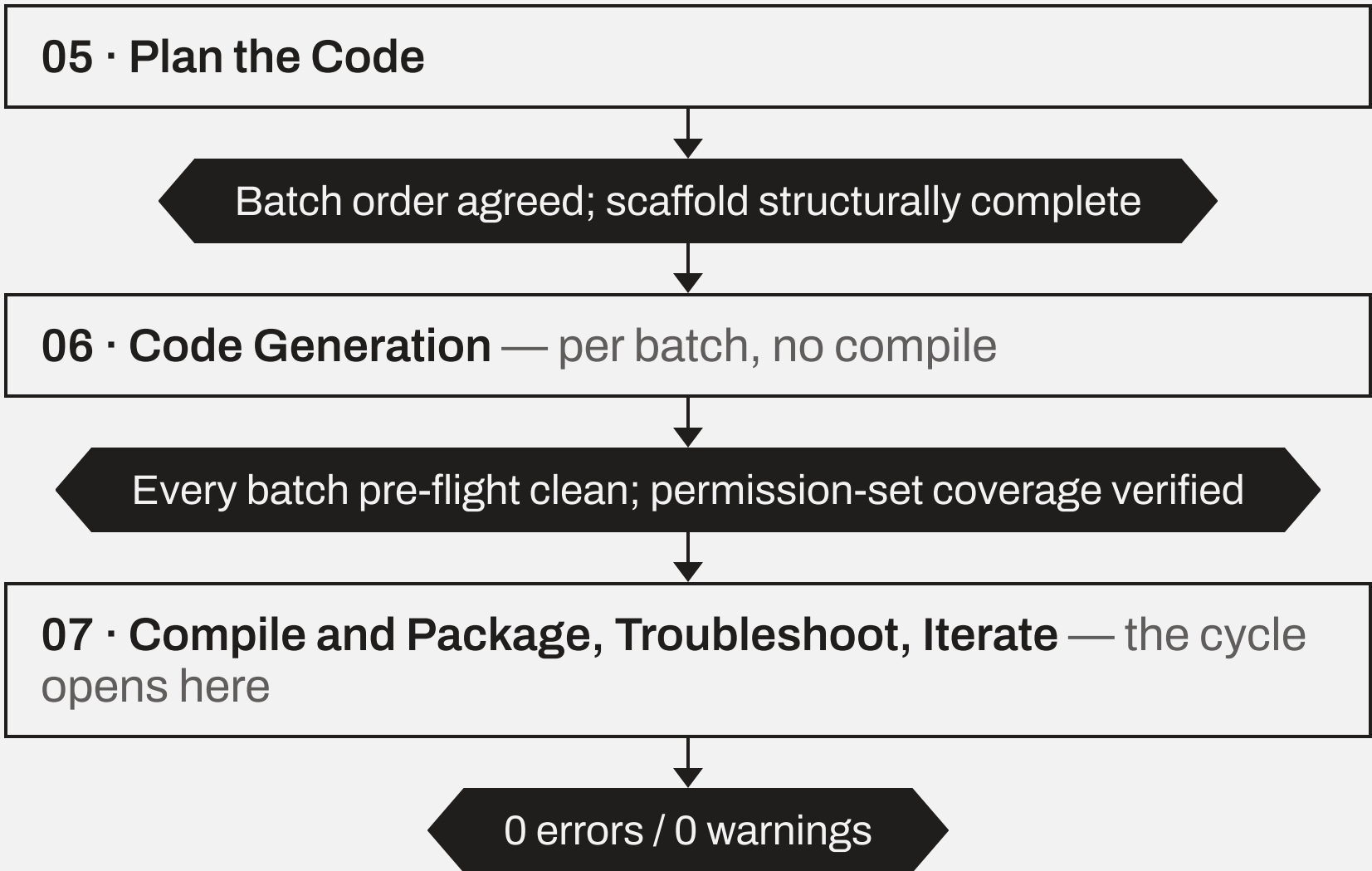
DESIGN



↓ to BUILD, Step 05

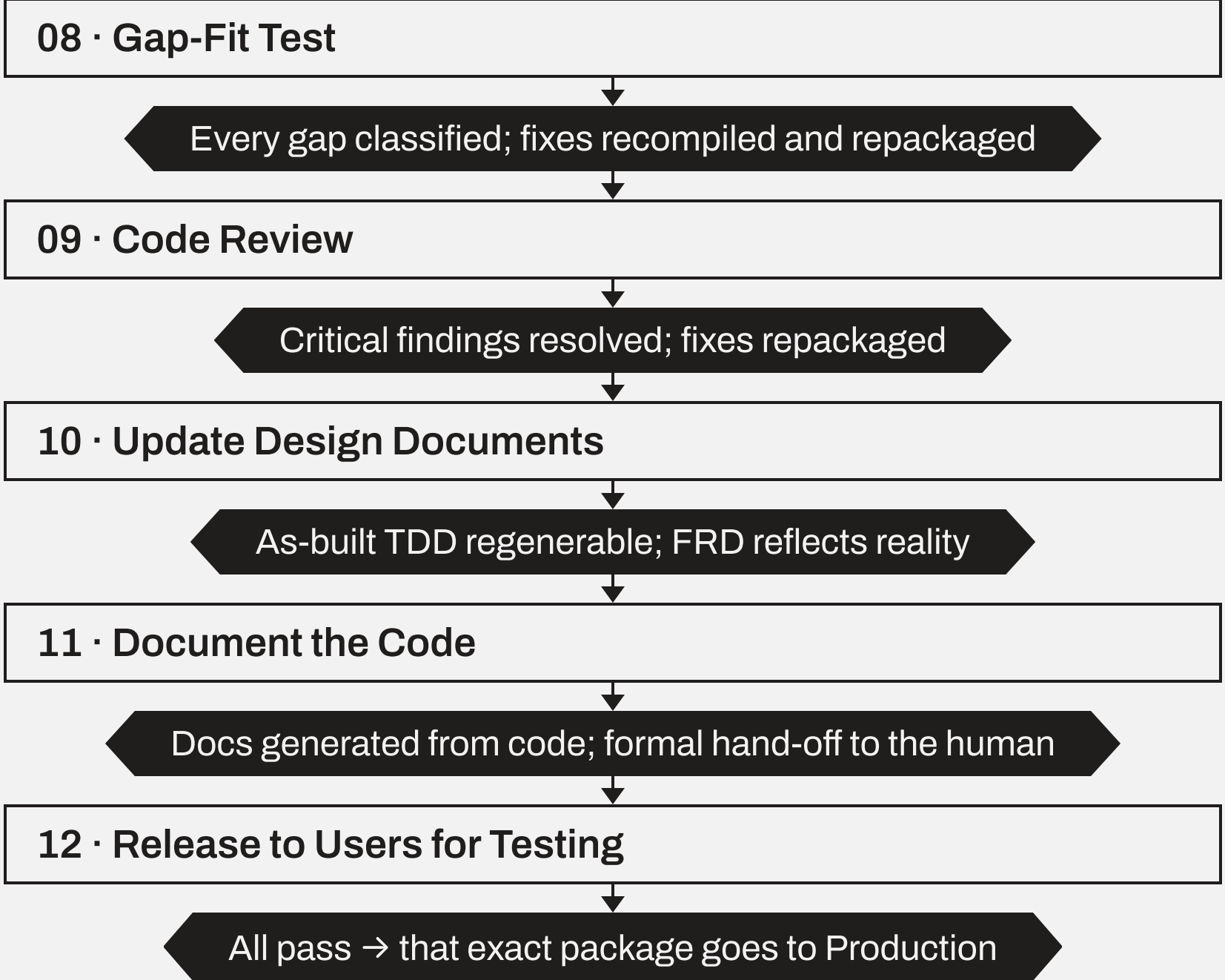
Every step, and the gate that closes it

BUILD

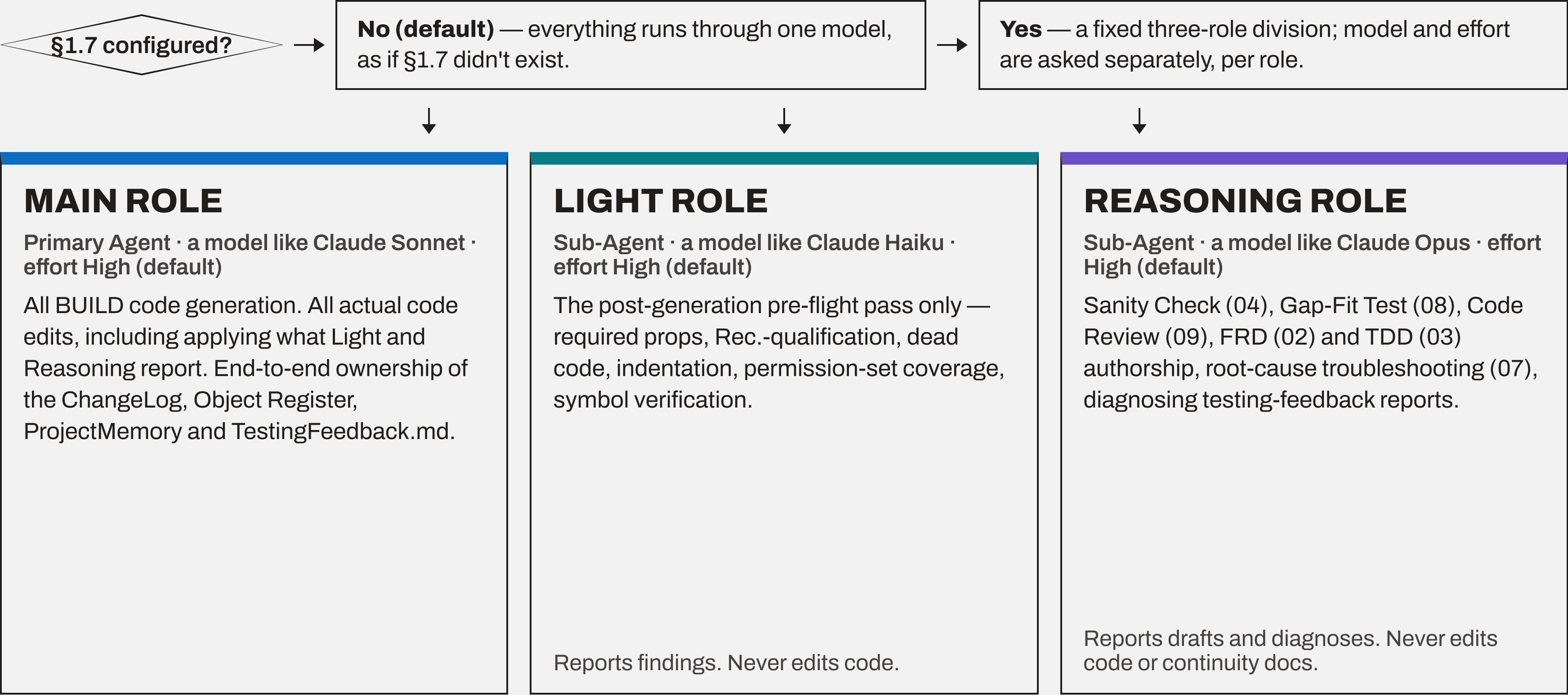


↓ to PROVE, Step 08

PROVE



Model & Effort Assignment (§1.7)



Light and Reasoning relay findings to Main ←-----→ **The codebase & continuity documents**

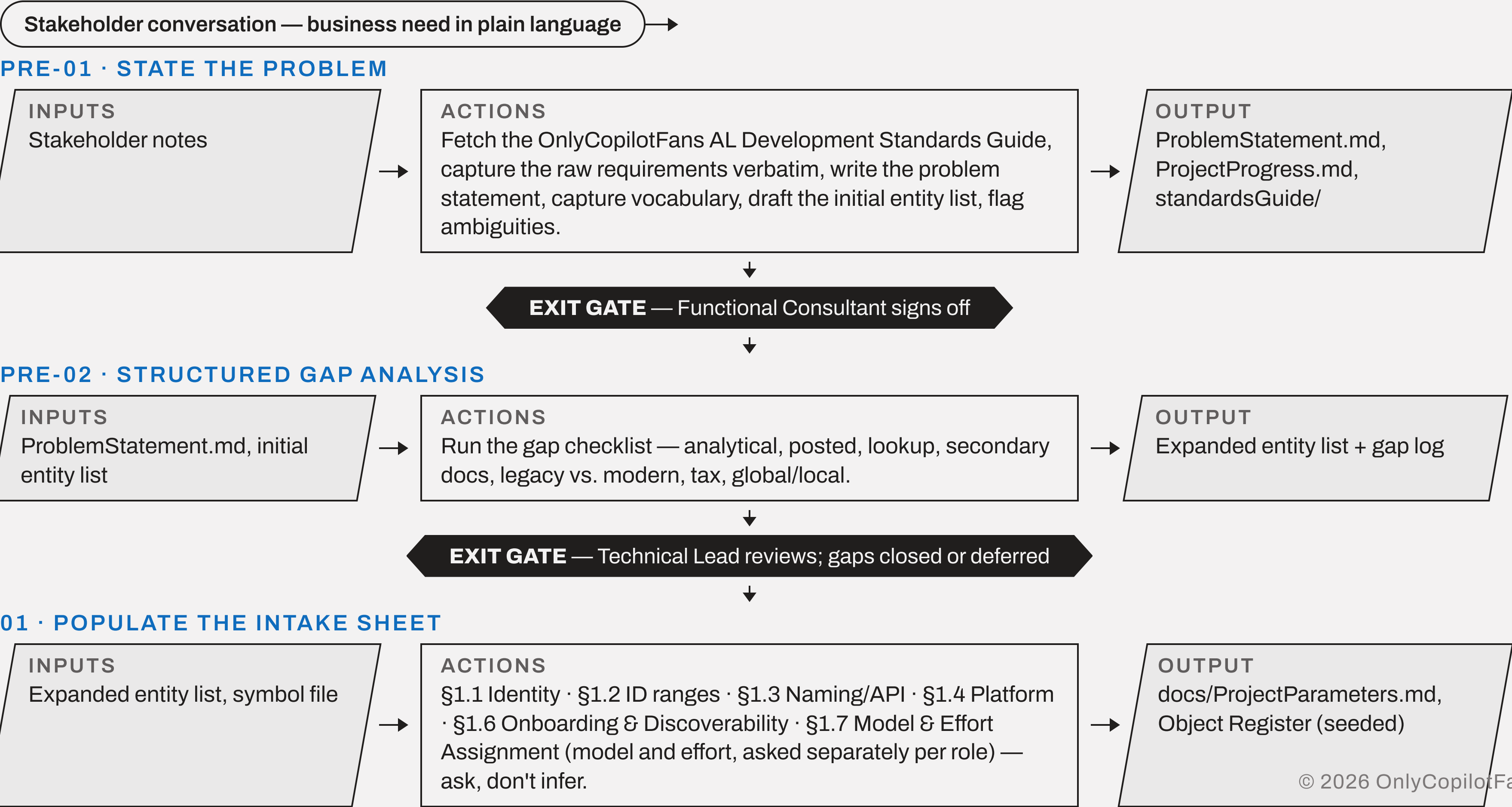
PHASE 01

DEFINE

Scope the problem and fill in the parameters. Ask, don't infer.

PRE-01 State the Problem · PRE-02 Structured Gap Analysis · 01 Populate the Intake Sheet

From stakeholder conversation to confirmed parameters



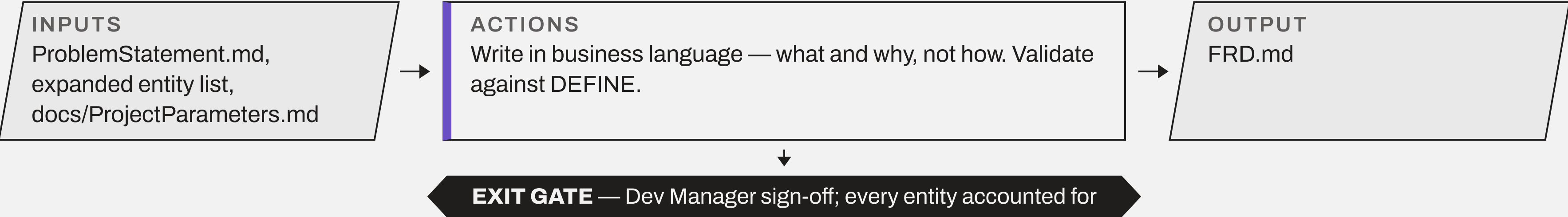
DESIGN

An FRD in business language, and a TDD complete enough to build from alone.

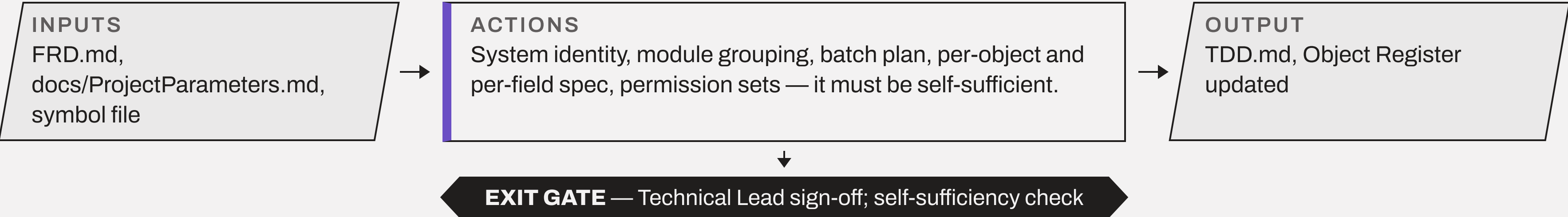
The Main role integrates every draft — saves the file, keeps ChangeLog and ProjectMemory bookkeeping — and carries it to the human for sign-off. The Reasoning role never owns sign-off.

Write what, then how — and check they agree

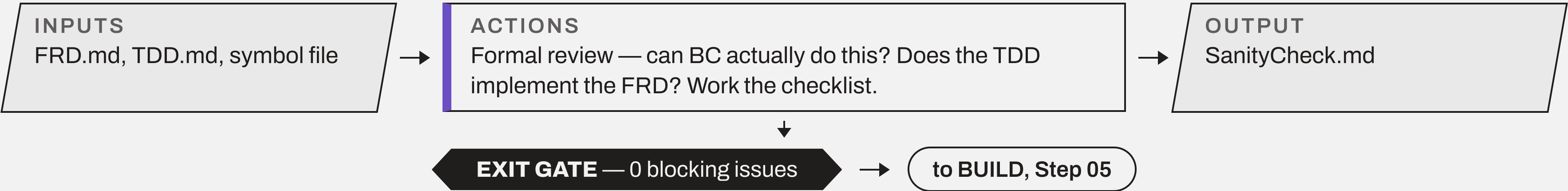
02 · CRAFT THE FRD REASONING ROLE



03 · CRAFT THE TDD REASONING ROLE



04 · SANITY CHECK AND VALIDATION REASONING ROLE

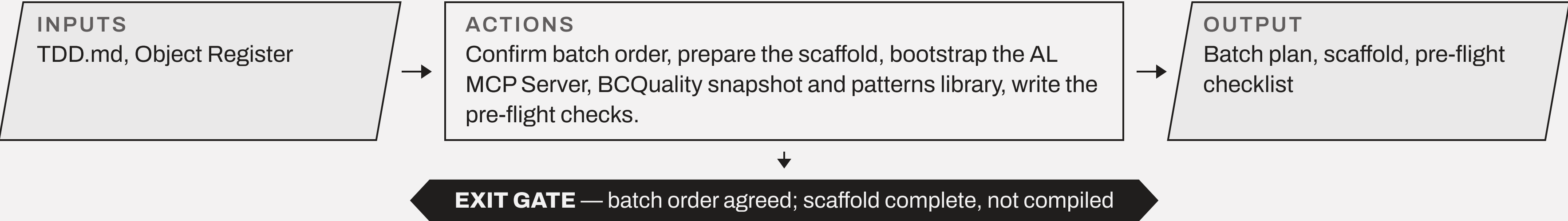


BUILD

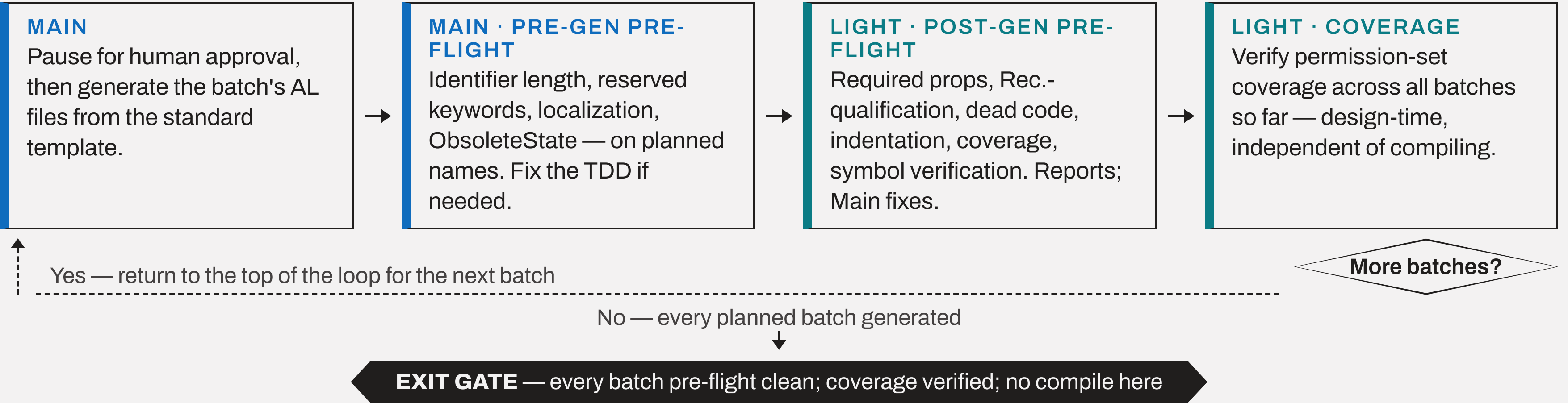
No per-batch compile. Symbol-verified lint instead, then one mandatory compile-and-package once every planned batch exists — after which packaging is the continuous rhythm.

Plan the batches, then generate them — without compiling

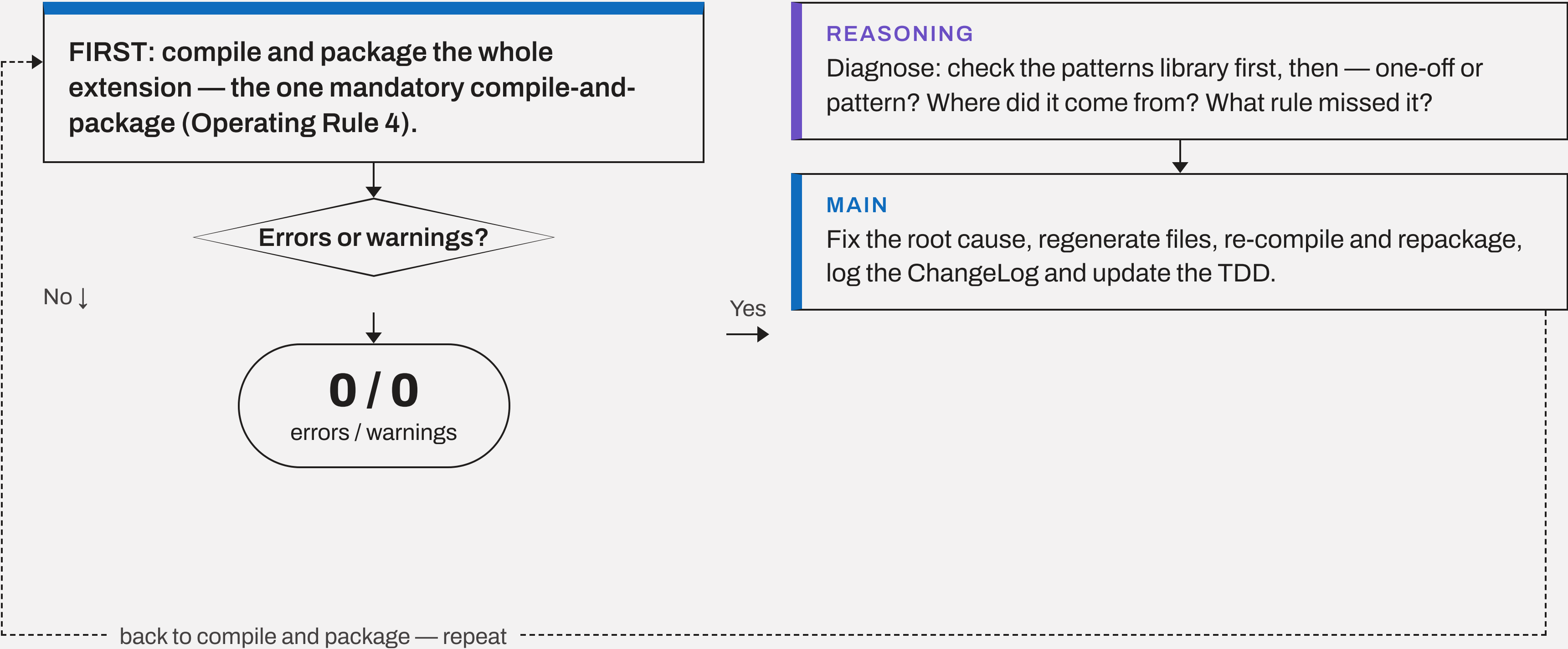
05 · PLAN THE CODE



06 · CODE GENERATION repeats once per batch



Compile and package, troubleshoot, iterate



EXIT GATE — extension compiles and packages clean; ChangeLog and TDD reconciled

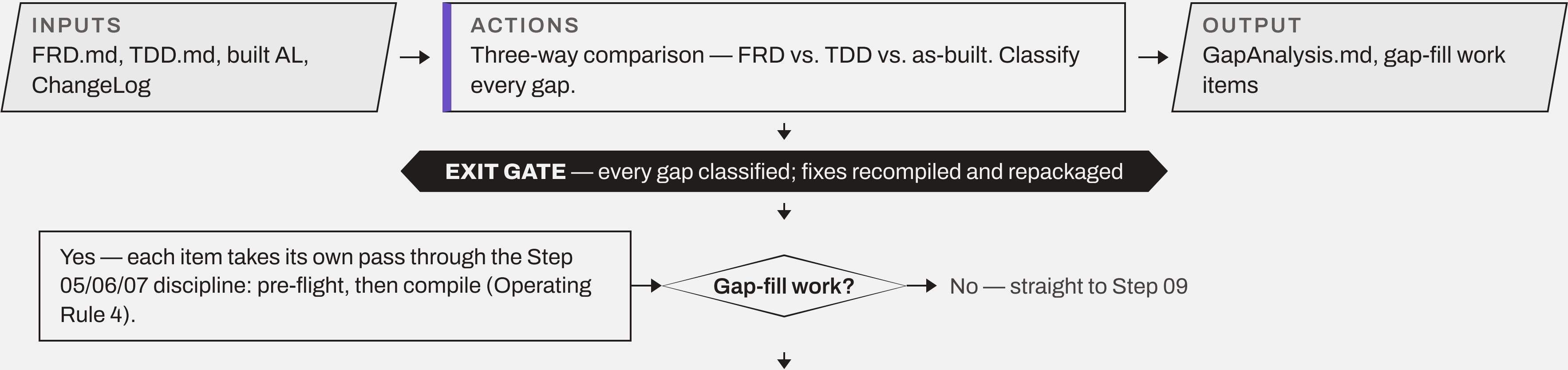
PROVE

Prove, review, document, release — until the package that was tested is the one in production.

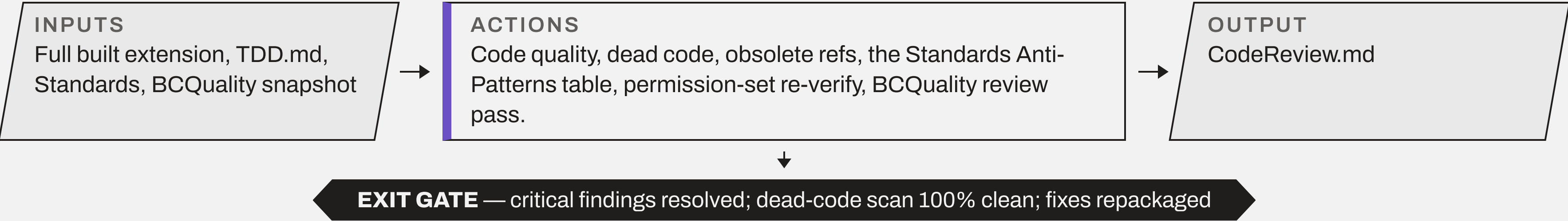
08 Gap-Fit Test · 09 Code Review · 10 Update Design Documents · 11 Document the Code · 12 Release to Users for Testing

Does the built thing match what was promised?

08 · GAP-FIT TEST, FIDELITY VALIDATION REASONING ROLE

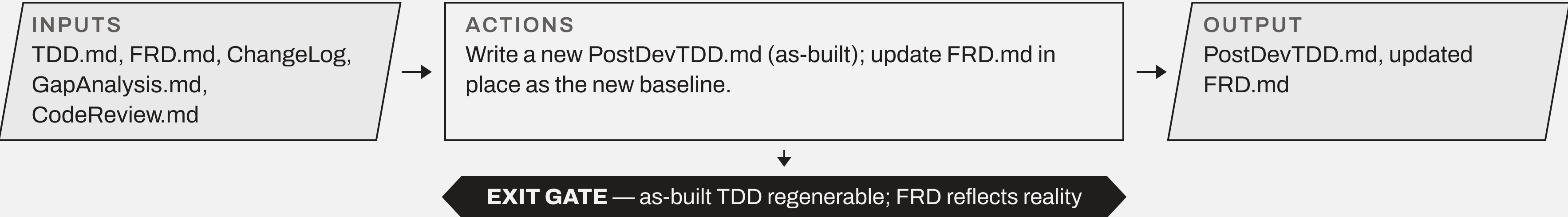


09 · CODE REVIEW REASONING ROLE

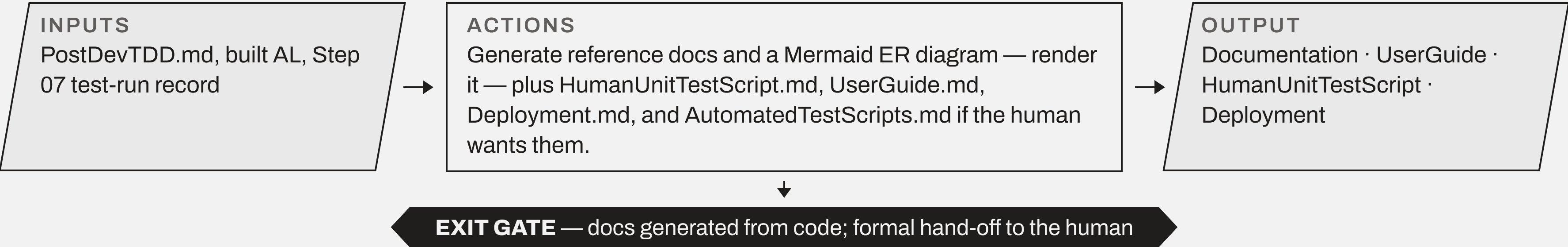


Leave the documentation behind, then let users test it

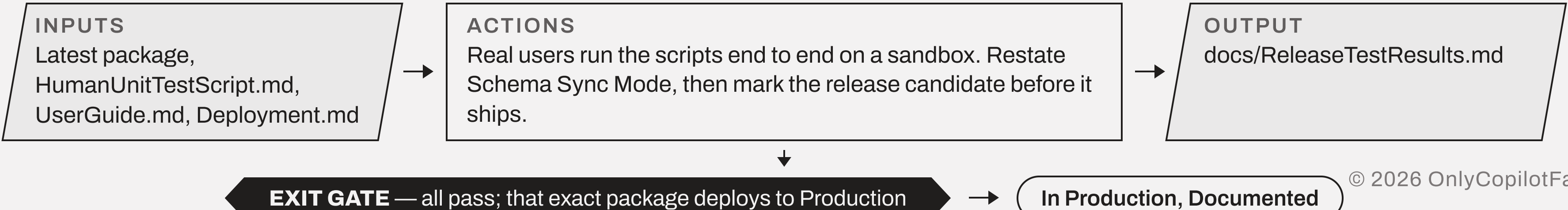
10 · UPDATE DESIGN DOCUMENTS



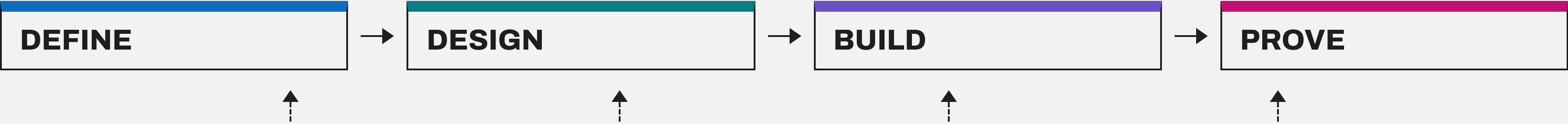
11 · DOCUMENT THE CODE



12 · RELEASE TO USERS FOR TESTING



ALL ALONG — continuous discipline



ALL ALONG EVERY PHASE, EVERY STEP

Document Nineteen required documents, kept current — never retroactively.	Track Changes Every FRD/TDD deviation logged before the next batch.	Retain Explanations Who decided, and why — not just the outcome.	Testing Feedback Log Verbatim findings from Step 07 onward, triaged explicitly.
Project Memory docs/ProjectMemory.md — an anchor, not a narrative.	Project Progress ProjectProgress.md in the root — one row per step, PRE-01 to 12.	Packaging & Versioning outputAppPackage/, never deleted; Schema Sync Mode flagged per build.	Repository Hygiene What never reaches the client's remote — and what always does.
AL MCP Server Bootstrap once; prefer its tools over ad hoc terminal use.	BCQuality Snapshot Fetched once, outside the project root; refreshed only on request.	AL Patterns Library Consulted before diagnosing from scratch at Step 07.	AL Standards Guide Fetched at PRE-01. Unreachable means stop and ask — never recall it.

Not every extension needs fourteen steps

LITE

7 steps · 4 documents

Roughly 5–10 AL files. One person driving, one AI model doing the work. No separate Dev Manager / Technical Lead / Functional Consultant sign-off roles.

FULL

14 steps · 19 documents

Past ~10 objects, multiple sign-off roles, work split across Main / Light / Reasoning models, or an extension heading to AppSource.

Lite reduces process, never the AL rules

Both editions fetch and apply the same OnlyCopilotFans AL Development Standards Guide. The same rules apply to a 5-file extension as to a 50-file one.

FRD + TDD + Sanity Check → DesignDoc.md	Testing Feedback Log + Roadmap → ChangeLog.md	Multi-model role split → dropped
---	---	---

Nothing is lost by switching later. **DesignDoc.md** maps directly onto the full framework's TDD, and **ChangeLog.md** carries straight over. One edition per project — never both at once.

Simple Enough for Functional Consultants. Robust Enough for Pro Developers.

Created by AJ Ansari, Microsoft MVP